

Intermediate (build the loop yourself) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

```
# react_loop_practice.py – Intermediate section
class MaxIterationsExceeded(Exception):
    pass

def run_agent(task, max_iterations=5):
    messages = [{"role": "user", "content": task}]

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=[tool_schema],
        )
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result = run_tool(call)
                messages.append(
                    {"role": "tool", "tool_call_id": call.id, "content": result},
                )
        else:
            return message.content

    message = "Agent ran out of steps without an answer: " + task
    raise MaxIterationsExceeded(message)
```

This version works correctly for one tool that always succeeds. It's missing type hints, and `run_tool()` here is assumed to never raise — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — one tool, minimal structure

```
# react_loop_practice.py – Intermediate section
def run_tool(call) -> str:
    import json
    try:
        args = json.loads(call.function.arguments)
        result = get_weather(**args)
        return str(result)
    except Exception as e:
        return f"Error: {e}"

def run_agent(task: str, max_iterations: int = 5) -> str:
    messages: list[dict] = [{"role": "user", "content": task}]

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=[weather_tool_schema],
        )
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result = run_tool(call)
                messages.append(
                    {"role": "tool", "tool_call_id": call.id, "content": result},
                )
        else:
            return message.content

    raise MaxIterationsExceeded(f"No answer after {max_iterations} steps")
```

Approach 2 — logging every step, and returning the full trace

```

# react_loop_practice.py – Intermediate section
from dataclasses import dataclass, field

class MaxIterationsExceeded(Exception):
    def __init__(self, message: str, steps: list[dict]) -> None:
        self.steps = steps
        super().__init__(message)

@dataclass
class AgentResult:
    final_answer: str
    steps: list[dict] = field(default_factory=list)

def run_agent(task: str, max_iterations: int = 5) -> AgentResult:
    messages: list[dict] = [{"role": "user", "content": task}]
    steps: list[dict] = []

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=[weather_tool_schema],
        )
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result = run_tool(call)
                messages.append(
                    {"role": "tool", "tool_call_id": call.id, "content": result},
                )
                steps.append({
                    "step": step,
                    "tool": call.function.name,
                    "arguments": call.function.arguments,
                    "result": result,
                })
            else:
                return AgentResult(final_answer=message.content, steps=steps)

    raise MaxIterationsExceeded(f"No answer after {max_iterations} steps", steps)

```

Difference from Basic: both add full type hints (`task: str` , `-> str` / `-> AgentResult`).

Approach 2 additionally records each step's tool, arguments, and result as it happens, and returns them alongside the answer — once the final answer is wrong, you have a real trail to look at instead of re-running with print statements sprinkled in by hand.

Approach 3 — a real multi-tool dispatch table, every tool call handled

Story: Approach 2 still hardcodes one tool (`get_weather`) by name inside `run_tool()` . The Build Task needs 3+ tools, chosen by the model at runtime — that means `run_tool()` has to look up *whichever* tool was requested, by name, from a table, not call one hardcoded function. **If not:** the Build Task's `agent.py` would be the first place you ever wrote a tool dispatch table, with no smaller version to build on.

```
# react_loop_practice.py – Intermediate section
import json
from dataclasses import dataclass, field

class MaxIterationsExceeded(Exception):
    def __init__(self, message: str, steps: list[dict]) -> None:
        self.steps = steps
        super().__init__(message)

@dataclass
class AgentResult:
    final_answer: str
    steps: list[dict] = field(default_factory=list)

TOOLS_BY_NAME = {
    "get_weather": get_weather,
    "celsius_to_fahrenheit": celsius_to_fahrenheit,
}

def run_tool(call) -> str:
    # why: looks the tool up by name instead of hardcoding one – this is
    # what lets the same function serve any number of registered tools.
    function = TOOLS_BY_NAME.get(call.function.name)
    if function is None:
        return f"Error: unknown tool {call.function.name}"
    try:
        args = json.loads(call.function.arguments)
        result = function(**args)
        return str(result)
```

```

except Exception as e:
    # how: never lets an exception escape – the model sees a clear
    # error string as the Observation instead of the program crashing.
    return f"Error: {e}"

def run_agent(task: str, max_iterations: int = 5) -> AgentResult:
    messages: list[dict] = [{"role": "user", "content": task}]
    steps: list[dict] = []

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=[weather_tool_schema, fahrenheit_tool_schema],
        )
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result = run_tool(call)
                tool_message = {
                    "role": "tool", "tool_call_id": call.id, "content": result
                }
                messages.append(tool_message)
                steps.append({
                    "step": step,
                    "tool": call.function.name,
                    "arguments": call.function.arguments,
                    "result": result,
                })
            else:
                return AgentResult(final_answer=message.content, steps=steps)

    raise MaxIterationsExceeded(f"No answer after {max_iterations} steps", steps)

```

Expected behavior if `get_weather` raises on a bad city name: `run_tool()` catches it, and the loop's next round sees `{"role": "tool", "content": "Error: city not found"}` as the Observation — the model gets a real chance to react (retry, ask for clarification, or say it can't answer), instead of the whole program crashing and every already-completed step being lost.

Difference from Approach 2: Approach 2 logs steps and handles one tool with a bare `try/except`. Approach 3 generalizes to a real multi-tool dispatch table (`T00LS_BY_NAME`) so `run_tool()` works for any registered tool, not just `get_weather` by name — this is what the Build Task's "3+ tools" requirement actually needs, and it never lets a tool's exception escape.

Which one should you actually write? Approach 3's shape — a dispatch table, a `run_tool()` that never lets an exception escape, and a returned `steps` log — is what the Build Task's `agent.py` should look like. Approach 1 is genuinely fine for a single quick script with one trusted tool; reach for Approach 3's shape the moment you have more than one tool, or any tool that touches the network, a file, or anything else that can fail. Worth knowing beyond this exercise: proving the step limit genuinely holds under adversarial conditions (a tool that always asks for another call) is something worth testing with a real automated test once you're writing `test_agent.py` for the Build Task — this exercise's job is building the loop, not testing it exhaustively.